SUBTRACK | JAVA DOCUMENTATION

Master branch | Java 21 | Spring Boot 3.5 | MySQL | React + TypeScript

OVERVIEW

SubTrack tracks recurring subscriptions, billing dates, payment activity, account balance, and notifications. This PDF documents the Java migration on master. Main is not modified.
Current Java features: registration/login, BCrypt password hashing, JWT creation, subscription CRUD/status, balance-backed payment recording, notifications, and health endpoint.
This is a migration baseline, not a production-ready payment or identity service.

ARCHITECTURE AND STACK

React + TypeScript + Vite client -> HTTP/JSON -> Spring Boot REST API -> Spring Data JPA -> MySQL.
Backend: Java 21, Spring Boot 3.5.6, Spring Web, Spring Data JPA, Jakarta Validation, Spring Security, Maven, Hibernate, MySQL Connector/J. Client: React, TypeScript, Vite, Axios, React Router, React Hook Form, Tailwind CSS, Recharts.

Source layout: Client/ frontend; Server/pom.xml Maven build; Server/src/main/java/com/subtrack/ contains config, controller, dto, entity, repository, security; Server/src/main/resources contains application properties; Documentation/ contains API and development notes.

DOMAIN MODEL

User: UUID, username, email, BCrypt password hash, balance, created timestamp, role.
Subscription: user UUID, name, amount, category, billing cycle/status, next billing timestamp.
Payment: user/subscription UUIDs, amount, status, payment timestamp, unique transaction reference.
Notification: owner, title/message/type, read state and creation metadata.
Money uses BigDecimal; timestamps use Instant. Billing cycles: Monthly, Quarterly, HalfYearly, Yearly.
Subscription statuses: Active, Paused, Cancelled. Payment statuses: PENDING, SUCCESS, FAILED.

LOCAL DEVELOPMENT

Prerequisites: JDK 21, Maven 3.9+, MySQL 8-compatible server, Node.js/npm compatible with Vite.

Clone and select Java branch:

```
git clone https://github.com/chauhan12harsh/Subtrack.git
cd Subtrack
git switch master
```

Create MySQL schema named subtrack or configure DB_URL. Hibernate ddl-auto=validate checks mappings against an existing schema; it does not create or migrate tables. No Flyway/Liquibase migration is currently included. Confirm schema mappings before startup.

ENVIRONMENT VARIABLES

SERVER_PORT HTTP port; default 8080
DB_URL JDBC URL; default jdbc:mysql://localhost:3306/subtrack
DB_USERNAME database user; default root
DB_PASSWORD database password; default empty
JWT_SECRET HS256 signing secret; minimum 32 characters for token creation
JWT_ISSUER issuer claim; default subtrack
JWT_AUDIENCE configured default subtrack-client; currently not emitted or validated

Example (use local values; never commit secrets):

```
export DB_URL='jdbc:mysql://localhost:3306/subtrack'
export DB_USERNAME='root'
export DB_PASSWORD='your-local-password'
export JWT_SECRET='replace-with-a-long-random-development-secret'
export SERVER_PORT=8080
```

Run backend: cd Server && mvn spring-boot:run
Run frontend in another terminal: cd Client && npm install && npm run dev
Health check: GET http://localhost:8080/health

SECURITY, PAYMENTS, ROADMAP

SECURITY WARNING

JWT creation exists, but bearer-token verification is not implemented. Spring Security currently permits requests. User-scoped endpoints trust caller-supplied X-User-Id, which can be spoofed. Do not expose the API to untrusted networks. Implement JWT signature/expiry/issuer validation, derive identity from authenticated principal, remove identity headers, configure CORS, and test access.

IDEMPOTENCY LIMITATIONS

The payment key is saved as a unique transaction reference and duplicate lookup returns an existing record. This is not fully concurrency-safe, does not verify same-key payload consistency, and has no dedicated durable idempotency ledger. Do not process real money. A robust design needs a scoped unique key, request fingerprint, persisted outcome, atomic payment/balance effects, locking or optimistic concurrency, and integration tests for simultaneous retries.

FRONTEND INTEGRATION

The client still has legacy API paths and a legacy base URL. Align routes, DTOs, token handling, response/error contracts, and CORS before expecting end-to-end operation.

SUGGESTED CHECKS (not verified as passing)

Backend: cd Server && mvn test
Frontend: cd Client && npm ci && npm run lint && npm run build

RELEASE CHECKLIST

[ ] JWT verification and principal-based ownership; remove X-User-Id trust
[ ] Versioned database migrations and reproducible schema setup
[ ] Durable, payload-aware, concurrency-safe idempotency and balance consistency
[ ] Align frontend routes/types and add contract/end-to-end tests
[ ] Port required profile/password/account and billing behavior
[ ] Unit, repository, API, security, DB integration, and concurrency tests
[ ] OpenAPI, consistent errors, logging, secrets, CORS, deployment hardening, verified builds

Related repository docs: README.md, Documentation/API.md, Documentation/DEVELOPMENT.md, Server/README.md. This PDF describes current master code and known gaps; it is not a production readiness review.